

M-Kristo App

Daily Devotions Platform — m-kristo.com

Deployment & Hosting Guide

Document code	BGT-MKR-DEP-001
Version	1.0
Date	12 August 2026
Status	Draft for Review
Classification	Confidential

Document Control

Field	Detail
Project	M-Kristo App (m-kristo.com)
Document	Deployment & Hosting Environment Guide
Document code	BGT-MKR-DEP-001
Version	1.0
Date	12 August 2026
Author	Philip Steven Chediel
Organisation	BlueGrid Technologies
Contact	www.bluegrid.co.tz +255 620 636 893
Related	BGT-MKR-SRS-001 (SRS), BGT-MKR-SDD-001 (SDD)

Revision History

Version	Date	Author	Description
1.0	12 August 2026	Philip Steven Chediel	Initial deployment & hosting baseline.

Table of Contents

1.	Introduction & Overview
2.	Environment Strategy (Dev, Staging, Production)
3.	Phase 1 — Local Development Setup
4.	Phase 2 — Backend Hosting (Django + PostgreSQL)
5.	Phase 3 — Third-Party Services Configuration
6.	Phase 4 — Mobile Build with Expo EAS
7.	Phase 5 — Publishing to Google Play
8.	Phase 6 — Publishing to the Apple App Store
9.	Phase 7 — CI/CD & Over-the-Air Updates
10.	Phase 8 — Monitoring, Backups & Maintenance
11.	Deployment Checklist

1. Introduction & Overview

1.1 Purpose

This guide describes, step by step, how the **M-Kristo App** moves from a developer workstation to live production. It covers the React Native (Expo) mobile client, the Django backend, the PostgreSQL database, supporting third-party services, and publishing to both the Google Play Store and the Apple App Store.

1.2 High-Level Flow

- **Develop** locally → run the Django API and the Expo client on a device/emulator.
- **Host the backend** → deploy Django + PostgreSQL to a production server over HTTPS.
- **Configure services** → SMS OTP, push, payments and storage.
- **Build the apps** → produce signed Android and iOS binaries with Expo EAS.
- **Publish** → submit to Google Play and the Apple App Store.
- **Operate** → CI/CD, OTA updates, monitoring and backups.

1.3 Accounts You Will Need

Account	Used for
Expo (EAS)	Building and submitting mobile binaries.
Google Play Console	Publishing the Android app (one-time USD 25 fee).
Apple Developer Program	Publishing the iOS app (annual USD 99 fee).
Cloud/VPS host	Hosting the Django backend and PostgreSQL.
Domain / DNS (m-kristo.com)	API subdomain and TLS certificate.
Firebase	Push notifications (FCM) and optional analytics.
SMS gateway	Sending OTP verification codes.
Payment provider	Subscription billing.

2. Environment Strategy

Three isolated environments are used so that changes are validated before reaching real users. Each has its own database, secrets and configuration.

Environment	Purpose	Backend URL (example)
Development	Local coding and debugging on the developer machine.	http://localhost:8000
Staging	Pre-release testing on real devices with production-like data.	https://staging-api.m-kristo.com
Production	Live environment used by end users.	https://api.m-kristo.com

The mobile client selects the correct API base URL per build profile (development / preview / production) so the same codebase targets any environment.

3. Phase 1 — Local Development Setup

Step 1.1 — Install prerequisites

- Install **Node.js (LTS)**, **Git**, and a package manager (npm or yarn).
- Install **Python 3.11+** and **pip** for the Django backend.
- Install **PostgreSQL** locally (or run it via Docker).
- Install the **Expo CLI** and the **Expo Go** app on a test phone, plus Android Studio / Xcode for emulators.

Step 1.2 — Set up the Django backend

- Clone the repository and create a virtual environment (*python -m venv .venv*).
- Install dependencies (*pip install -r requirements.txt*).
- Create a **.env** file with database credentials and secret keys.
- Run migrations (*python manage.py migrate*) and create an admin user (*createsuperuser*).
- Start the API (*python manage.py runserver*).

Step 1.3 — Set up the Expo client

- Install packages (*npm install*) in the mobile project.
- Point the development API base URL at your machine's LAN IP so a physical phone can reach it.
- Start the bundler (*npm expo start*) and open the app in Expo Go or an emulator.
- Verify signup, OTP, login, Bible reading and daily content against the local API.

4. Phase 2 — Backend Hosting (Django + PostgreSQL)

The backend is hosted on a Linux server (managed platform or VPS) behind HTTPS, with a managed PostgreSQL database.

Step 2.1 — Provision the server & database

- Create a Linux server (or a managed app platform) for the Django app.
- Provision a **managed PostgreSQL** instance and record its connection string.
- Create a DNS **A/CNAME** record for **api.m-kristo.com** pointing to the server.

Step 2.2 — Configure the production app

- Set environment variables: `SECRET_KEY`, `DEBUG=False`, `ALLOWED_HOSTS`, `DATABASE_URL` and service keys.
- Collect static files (*`python manage.py collectstatic`*).
- Run migrations against the production database.
- Serve the app with **Gunicorn/Uvicorn** behind **Nginx** as a reverse proxy (or use the platform's runtime).

Step 2.3 — Enable HTTPS

- Obtain a TLS certificate (e.g. Let's Encrypt) for **api.m-kristo.com**.
- Force HTTPS and enable HSTS; the mobile app must only talk to the app over TLS.

Step 2.4 — Verify the deployment

- Hit a health-check endpoint and confirm a JSON response over HTTPS.
- Confirm the Django admin loads and the database is reachable.

5. Phase 3 — Third-Party Services Configuration

Service	Setup steps
SMS OTP gateway	Create an account, obtain API credentials, set the sender ID, and store keys as backend env vars for phone verification.
Firebase (FCM)	Create a Firebase project; add Android and iOS apps; download google-services.json and GoogleService-Info.plist; add the server key to the backend.
Push (Expo/EAS option)	Alternatively use Expo push tokens; store tokens per device and send via the Expo push service.
Payment provider	Create products for the monthly and annual plans; configure webhooks to the backend to activate subscriptions.
Media/storage	Configure object storage or media hosting for profile pictures and content assets.

Keep separate credentials for staging and production, and never commit secrets to source control.

6. Phase 4 — Mobile Build with Expo EAS

Step 4.1 — Prepare the project

- Install the EAS CLI (*npm install -g eas-cli*) and log in (*eas login*).
- Set app identity in **app.json/app.config**: name, slug, version, Android *package* and iOS *bundleIdentifier*, icons and splash screen.
- Add the app icon and branding assets that match the M-Kristo look.

Step 4.2 — Configure build profiles

- Run *eas build:configure* to create **eas.json**.
- Define **development**, **preview** and **production** profiles, each pointing to the correct API URL.

Step 4.3 — Manage signing credentials

- Let EAS generate and store the Android **keystore** (or upload your own).
- For iOS, sign in with your Apple Developer account so EAS can manage the distribution certificate and provisioning profile.

Step 4.4 — Build the binaries

- Android: *eas build --platform android --profile production* (produces an **.aab**).
- iOS: *eas build --platform ios --profile production* (produces an **.ipa**).
- Download or let EAS store the artifacts for submission.

7. Phase 5 — Publishing to Google Play

Step 5.1 — Prepare the listing

- Create the app in the **Google Play Console** and pay the one-time registration fee.
- Complete the store listing: title, short/full description (Swahili and English), screenshots, feature graphic and app icon.
- Fill the content rating, data-safety, privacy policy and target audience forms.

Step 5.2 — Upload and test

- Upload the **.aab** (manually or via *eas submit --platform android*).
- Release first to an **internal testing** track and verify on real devices.
- Promote to **closed/open testing** as needed.

Step 5.3 — Production release

- Promote the tested build to **Production**.
- Submit for Google review; once approved the app goes live on the Play Store.
- Use **staged rollout** to release to a percentage of users first.

8. Phase 6 — Publishing to the Apple App Store

Step 6.1 — Prepare in App Store Connect

- Enrol in the **Apple Developer Program** (annual fee).
- Create the app record in **App Store Connect** with the matching *bundleIdentifier*.
- Complete the listing: name, subtitle, description (Swahili and English), keywords, screenshots and privacy details.

Step 6.2 — Upload and TestFlight

- Submit the **.ipa** with `eas submit --platform ios` (uploads to App Store Connect).
- Distribute the build via **TestFlight** to internal/external testers and validate.

Step 6.3 — Review and release

- Attach the build to a version and submit for **App Review**.
- Provide a demo account (with a verified phone/OTP path) and clear notes for reviewers.
- On approval, release manually or automatically to the App Store.

9. Phase 7 — CI/CD & Over-the-Air Updates

9.1 Continuous Integration / Delivery

- On each push, run automated tests and linting for both backend and client.
- Auto-deploy the backend to **staging** on merge; deploy to **production** on tagged releases.
- Trigger EAS builds automatically for release branches.

9.2 Over-the-Air (OTA) Updates

- Use **EAS Update** to push JavaScript/asset fixes without a full store submission.
- Reserve store submissions for native changes (new permissions, SDK upgrades, icons).
- Map update channels to build profiles (preview vs production).

10. Phase 8 — Monitoring, Backups & Maintenance

Area	Practice
Logging	Centralised backend logs with error levels and request tracing.
Crash reporting	Capture client crashes and ANRs (e.g. Sentry/Crashlytics).
Uptime	Health-check monitoring and alerting for the API domain.
Backups	Automated daily PostgreSQL backups with periodic restore tests.
Security updates	Regular dependency and OS patching; rotate secrets periodically.
Performance	Monitor API latency and database queries; scale instances as usage grows.

11. Deployment Checklist

Pre-Launch

- Production backend live on HTTPS at **api.m-kristo.com**.
- Database migrated and backed up; admin access confirmed.
- SMS OTP, push, and payment services configured with production keys.
- Store listings complete in Swahili and English with screenshots.
- Privacy policy and data-safety declarations published.
- Demo/test account prepared for store reviewers.

Launch

- Android production build approved and rolled out on Google Play.
- iOS build approved and released on the App Store.
- Push notifications and subscription purchase verified end-to-end in production.

Post-Launch

- Monitoring and crash reporting confirmed to be receiving data.
- OTA update channel tested with a minor fix.
- Backup restore verified.

Prepared by Philip Steven Chediel, BlueGrid Technologies — www.bluegrid.co.tz — +255 620 636 893.